Kingdom of Saudi Arabia
Ministry of Education
University College of computer
Department of Computer Science

المملكة العربية السعودية
وزارة التعليم
كلية الحاسب
قسم علوم الحاسب

ONLINE FOOD ORDERING SYSTEM

CS342- Project

SUBMITTED BY:

Lamis Alorini 431201162
Shwaa Alrashidi 431201618
Amsha Al-rashidi 431201367
Joud Al-hoshani 431201376
Nada Al-khiari 431201359
Waad Ali 432205282

SUPERVISED BY:

Hoba Alsultan

TABLE OF CONTENT

Content	Pages
Introduction	3
GUI Elements	4
Exception Handling	8
Database Connection	9
Testing and Validation	10
Conclusion	11

INTRODUCTION

The Online Food Ordering System is a application developed in Java using Swing for the GUI and MySQL for the backend(database). threr are three types of users:Customers,Restaurant,Admins. It allows customers to register, log in, browse restaurants, select dishes, place orders, and make payments,and provide feedback. Restaurant owners can manage menus and orders, while admins oversee all users, restaurants, and order activities. The system provides order tracking and notification capabilities,and customer feedback functionality.

The report covers all aspects of the project, including the user interface design, data management, error handling, and testing to ensure the system operates efficiently and smoothly.

GUI ELEMENTS

The app uses Swing components to build the interface:
Key GUI Forms:
LoginForm:
Fields: EmailField(JTextField), PasswordField(JPasswordField)
Buttons: loginButton, registerButton, forgotPasswordButton(JButton)
label:Email,Password.
In login window:
Email
Password
Register
Login
ForgotPassword?
Registration Form:
Fields: nameField,EmailField(JTextField), PasswordField(JPasswordField) User Role (customer/restaurant/admin).
ComboBox: userTypeBox.
Buttons: registerButton.
label:Name,Email,Password.
Automatically adds new restaurants to the Restaurants table if the role is "restaurant".
In registration window:
name
email
password
User Role
register

GUI ELEMENTS

RetaurantListingForm:
Table:restaurantTable

RestaurantDashboard:
TextField dishNameField,dishPriceField, dishDescriptionField;
Table: dishesTable.
DefaultTableModel :tableModel.
Connection :conn.
Label:Dish Name,Dish Price,Dish Description.

ReviewAndTrackingForm:
ComboBox:orderComboBox,ratingComboBox.
TextArea :commentArea,previousReviewsArea.
Button :submitButton, exitButton;
Logger :logger
Allows customers to select an order, rate , and leave comments.
Inserts data into the Reviews table.

GUI ELEMENTS

OrderTrackingForm:
Table: ordersTable
Button: refreshButton, updateStatusButton, deleteOrderButton.
DefaultTableModel :tableModel.
Timer :timer.
TextField :orderIdField.
ComboBox:statusComboBox.
Shows all orders.
Allows status updates (pending → preparing → delivered).
Automatically refreshes every 60 seconds using Timer.

AdminDashboard:
Button:viewUsersBtn, deleteUserBtn, updateUserBtn,viewRestaurantsBtn, deleteRestaurantBtn, updateRestaurantBtn,trackOrdersBtn, viewFilterRestaurantsBtn,logoutButton, exitButton.
TextArea: outputArea.
Label :adminLabel.
Logger :logger.
Contains buttons to view, update, or delete users/restaurants.
Integrates buttons to open tracking and filtering forms.

GUI ELEMENTS

CartForm:
Table: cartTable.
DefaultTableModel :tableModel.
Label :totalLabel.

CustomerDashboard:
Panel: contentPanel.
DefaultTableModel: ordersModel.
Label:welome

DishListForm:
DefaultTableModel :tableModel.
Table: dishTable.
Button: refreshButton.

ForgotPasswordForm:
TextField: emailField.
Button :resetPasswordButton.
Label:email.

EXCEPTION HANDLING

Exception handling is vital to prevent crashes and inform the user of errors.
Types of Exceptions Handled:
SQLExceptions: When something goes wrong with the database (connection failure, invalid queries).
ClassNotFoundException: If JDBC driver is not found.
Validation Failures:
1.When users leave fields empty
.on log in when field email or password are empty
.On Register,name or email password user role is empty.
2. enter invalid data (weak password or email).
try-catch blocks wrap every database operation.
If an error occurs, the program logs it using Logger and shows a pop-up message (JOptionPane) like:
“Failed to load dishes”
“Invalid email or password”
In registration, checks ensure:
Email format is valid.
Password includes upper/lowercase, digits, and symbols.

DATABASE CONNECTION

The integration of a database into an application is a crucial step in
developing any robust software. For our application, we used MySQL as the database management system and connected it through the Services tab in NetBeans. This approach streamlined the development process by providing visual database management tools and simplifying debugging
Setting Up MySQL;
Before connecting to the database, we ensured the following:
MySQL server was installed and running on our machine.
our database was created, containing relevant our tables.
A MySQL user account (root) with appropriate privileges was configured for accessing the database.

All database interactions go through a central class:
DatabaseConnection class:
Contains a static getConnection() method.
Connects to the MySQL database:
DriverManager.getConnection("jdbc:mysql://localhost:3306/FoodO deringSystem", "root", "your_password");
Used by all other forms.
Checks if the driver is loaded and prints a success/failure message.
Database Tables:
Users: Stores user info (email, password, role)
Restaurants: Restaurant profiles and locations
Dishes: Menu items
Orders, Cart, OrderItems: Handle customer orders
Reviews: Restaurants ratings and comments
Notifications: For order status updates
Payments: Simulates transactions

TESTING AND VALIDATION

Testing ensures that each part of the system works correctly and securely.
Validation:
Validates(login) credentials against the database.

Registration form:
Validates(Registration) email format and password strength.
Validates empty fields
Uses regex for email format
Ensures strong password
Review form:
Validates restaurant and rating are selected
Order form:
Ensures cart is not empty before submitting

Testing Approaches:
Manual UI Testing:
Users interact with buttons, forms, and data displays.

Database Testing:
Data inserted or updated is checked directly in MySQL.

Error Simulation:
Entering wrong passwords or email shows expected errors.

Refresh Timers:
Order tracking refreshes every minute automatically to test real-time updates.

CONCLUSION

This project effectively demonstrates Java application integrating GUI, database.
Key Successes:
User Roles: Clearly separated functionalities for customers, restaurants, and admins. and GUI Built using Java Swing with responsive components.Database-Driven Fully integrated with MySQL for data storage and retrieval.Exception Handling: Protects against user and system errors.
issue
We encountered problems linking data.